

Document number: P0757R0
Date: 2017-09-10
Project: ISO JTC1/SC22/WG21, Programming Language C++
Audience: Library Evolution Working Group
Reply to: Arthur O'Dwyer <arthur.j.odwyer@gmail.com>

regex_iterator should be iterable

- [1. Introduction and motivation](#)
- [2. Member begin\(\) versus nonmember begin\(\)](#)
- [3. Alternative syntax ideas](#)
- [4. Proposed wording for C++20](#)
 - [28.4 Header <regex> synopsis \[re.syn\]](#)
 - [28.12 Regular expression iterators \[re.iter\]](#)
- [5. References](#)

1. Introduction and motivation

The C++17 filesystem library introduces a `directory_iterator` that can be used as follows: [\[directory_iterator\]](#)

```
#include <fstream>
#include <iostream>
#include <filesystem>
namespace fs = std::filesystem;

int main()
{
    fs::create_directories("sandbox/a/b");
    std::ofstream("sandbox/file1.txt");
    std::ofstream("sandbox/file2.txt");
    for (auto& p : fs::directory_iterator("sandbox")) {
        std::cout << p << '\n';
    }
    fs::remove_all("sandbox");
}
```

The `directory_iterator` object is both an iterator and an iterable, by the simple mechanism of providing an overload of `begin()` that returns `*this` and an overload of `end()` that returns `{}`.

Since C++11, the standard library has also provided a `regex_iterator` without this convenience feature. Its example usage on cppreference.com [\[regex_iterator\]](#) is:

```
#include <iostream>
#include <regex>
#include <string>

int main()
{
    const std::string s = "Quick brown fox.";

    std::regex words_regex("[^\\s]+");
    auto words_begin =
        std::sregex_iterator(s.begin(), s.end(), words_regex);
    auto words_end = std::sregex_iterator();

    for (std::sregex_iterator i = words_begin; i != words_end; ++i) {
```

```

        std::smatch match = *i;
        std::string match_str = match.str();
        std::cout << match_str << '\n';
    }
}

```

cppreference.com also includes a warning note:

It is the programmer's responsibility to ensure that the `std::basic_regex` object passed to the iterator's constructor outlives the iterator. Because the iterator stores a pointer to the regex, incrementing the iterator after the regex was destroyed accesses a dangling pointer.

This warning is apparently necessary in the case of `regex_iterator`, even though the same reasoning ought to apply to e.g. `std::istream_iterator` (it must not outlive its `istream`) and even `std::list::iterator` (it must not outlive its `list`). The reason we don't feel the need to warn people about lifetime bugs with `list` is that we don't often keep `std::list::iterator` objects alive outside of their natural (for-loop) scope. The "iterable iterator" idiom as seen in `directory_iterator` can help to keep iterators localized into tight scopes.

I propose that the following code should be well-formed:

```

#include <iostream>
#include <regex>
#include <string>

int main()
{
    const std::string s = "Quick brown fox.";

    std::regex words_regex("[^\\s]+");

    for (auto& match : std::sregex_iterator(s.begin(), s.end(), words_regex)) {
        std::string match_str = match.str();
        std::cout << match_str << '\n';
    }
}

```

2. Member `begin()` versus nonmember `begin()`

Why does `directory_iterator` provide `begin()` and `end()` as free functions instead of member functions? It's a mystery to me. But I'm content to follow its precedent, rather than propose member `begin()` and `end()` functions for `regex_iterator`.

Incidentally, although it's too late to change it now, I personally believe that both of these iterators would have benefited from a different structure; that is, rather than the syntax

```

for (auto& p : fs::directory_iterator("sandbox")) { ... }
for (auto& m : regex_iterator(begin(s), end(s), r)) { ... }

```

I would have preferred a syntax that split up the "iterable range" object from the "iterator" object, like this:

```

for (auto& p : fs::walk_directory("sandbox")) { ... }
for (auto& m : regex_iterate(s, r)) { ... }

```

3. Alternative syntax ideas

cppreference uses this style for its loop:

```

auto B = std::sregex_iterator(b, e, rx);
auto E = std::sregex_iterator();
for (auto it = B; it != E; ++it) {
    auto m = *it;
    // use m[0]
}

```

This can be expressed more compactly today as:

```

for (std::sregex_iterator it(b, e, rx); it != std::sregex_iterator{}; ++it) {
    auto m = *it;
    // use m[0]
}

```

The infelicity with that loop is the repetition of the name `std::sregex_iterator`, which would be avoidable even without the present proposal if we were to standardize an operator `bool` for `regex_iterator`. Thus:

```

// Assuming that regex_iterator were convertible to bool with the obvious semantics
for (std::sregex_iterator it(b, e, rx); it; ++it) {
    auto m = *it;
    // use m[0]
}

```

Another alternative today is:

```

auto it = std::sregex_iterator(b, e, rx);
while (it != std::sregex_iterator{}) {
    auto m = *it++;
    // use m[0]
}

```

This feels more palatable than the for-loop above, because even though it has the same number of repetitions of `sregex_iterator`, at least they're not both on the same line of code! However, practical experience shows that it is far too easy to write `*it` instead of `*it++`, which leads to an infinite loop. We should try to channel programmers into the common looping idioms where possible. Therefore I propose that `regex_iterator` should be made iterable with the ranged for-loop.

Should `regex_iterator`, `regex_token_iterator`, `directory_iterator`, `istream_iterator`, and `istreambuf_iterator` all be treated in roughly the same way?

Should these iterators be convertible to `bool`? (Right now none of them is.)

Should these iterators be iterable? (Right now `directory_iterator` is iterable but the others are not.)

4. Proposed wording for C++20

The wording in this section is relative to WG21 draft N4618 [\[N4618\]](#), that is, the current draft of the C++17 standard. The new wording is modeled on the existing wording in N4618 section 27.10.13.2 `[directory_iterator.nonmembers]`.

28.4 Header `<regex>` synopsis `[re.syn]`

Edit paragraph 1 as follows.

```

// 28.12.1, class template regex_iterator
template <class BidirectionalIterator,
class charT = typename iterator_traits<
BidirectionalIterator>::value_type,
class traits = regex_traits<charT>>

```

```

class regex_iterator;

using cregex_iterator = regex_iterator<const char*>;
using wcregex_iterator = regex_iterator<const wchar_t*>;
using sregex_iterator = regex_iterator<string::const_iterator>;
using wsregex_iterator = regex_iterator<wstring::const_iterator>;

// 28.12.2, range access for regex iterator
template<class B, class C, class T>
regex_iterator<B, C, T> begin(regex_iterator<B, C, T>) noexcept;
template<class B, class C, class T>
regex_iterator<B, C, T> end(const regex_iterator<B, C, T>&) noexcept;

// 28.12.2 28.12.3, class template regex_token_iterator
template <class BidirectionalIterator,
class charT = typename iterator_traits<
BidirectionalIterator>::value_type,
class traits = regex_traits<charT>>
class regex_token_iterator;

using cregex_token_iterator = regex_token_iterator<const char*>;
using wcregex_token_iterator = regex_token_iterator<const wchar_t*>;
using sregex_token_iterator = regex_token_iterator<string::const_iterator>;
using wsregex_token_iterator = regex_token_iterator<wstring::const_iterator>;

// 28.12.4, range access for regex token iterator
template<class B, class C, class T>
regex_token_iterator<B, C, T> begin(regex_token_iterator<B, C, T>) noexcept;
template<class B, class C, class T>
regex_token_iterator<B, C, T> end(const regex_token_iterator<B, C, T>&)
noexcept;

```

28.12 Regular expression iterators [re.iter]

Renumber section 28.12.2 [re.tokiter] to 28.12.3. Insert a new section following section 28.12.1 as follows.

28.12.2 regex_iterator non-member functions [re.iter.nonmember]

1. These functions enable range access for regex_iterator.

```

template<class B, class C, class T>
regex_iterator<B, C, T> begin(regex_iterator<B, C, T> iter) noexcept;

```

Returns: iter.

```

template<class B, class C, class T>
regex_iterator<B, C, T> end(const regex_iterator<B, C, T>&) noexcept;

```

Returns: regex_iterator<B, C, T>().

Insert a new section after the renumbered section 28.12.3 [re.tokiter] as follows.

28.12.4 regex token iterator non-member functions [re.tokenizer.nonmember]

1. These functions enable range access for regex token iterator.

```
template<class B, class C, class T>  
regex_token_iterator<B, C, T> begin(regex_token_iterator<B, C, T> iter) noexcept;
```

Returns: iter.

```
template<class B, class C, class T>  
regex_token_iterator<B, C, T> end(const regex_token_iterator<B, C, T>&) noexcept;
```

Returns: regex_token_iterator<B, C, T>().

5. References

[directory_iterator]

"std::filesystem::directory_iterator" on cppreference.com.

http://en.cppreference.com/w/cpp/filesystem/directory_iterator.

[N4618]

"Working Draft, Standard for Programming Language C++" (Nov 2016).

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4618.pdf>.

[regex_iterator]

"std::regex_iterator" on cppreference.com.

http://en.cppreference.com/w/cpp/regex/regex_iterator.